《深度学习》

Chap4 函数拟合实验报告

基于纯 NumPy 与 TensorFlow 框架的对比研究

学号：2351407

姓名：楼思含

专业：数据科学与大数据技术

2026 年 3 月 16 日

目　录

一、实验目的.....1
二、函数定义.....1
2.1 混合特征的深度融合.....1
2.2 非线性映射能力的严苛检验.....2
三、数据采集与隔离规范.....2
3.1 全量样本的无偏生成.....2
3.2 基于无放回抽样的绝对互斥切分.....2
四、模型描述.....3
4.1 拓扑结构搭建与非线性映射机理.....3
4.2 目标驱动函数与回归输出配置.....4
4.3 基于纯 NumPy 的底层微积分手工推演.....4
4.4 基于 TensorFlow 的计算图抽象与自动求导.....5
五、拟合效果对比与实验结论.....6
5.1 高度一致的最优化收敛轨迹.....6
5.2 解构复杂规律的视觉映射.....9
5.3 卓越泛化性能的证实.....10
5.4 数理同归.....12
六、参考文献.....13

一、实验目的

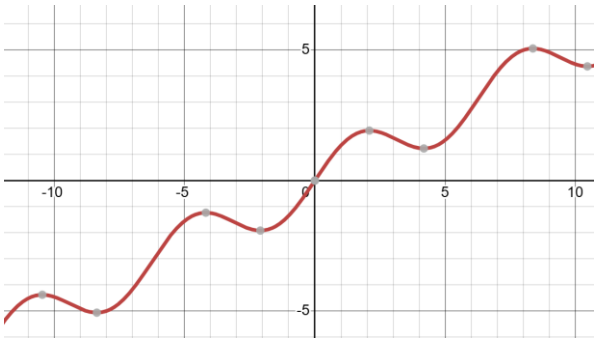
理论与大量实验证明，一个仅包含两层隐藏层且使用 ReLU（修正线性单元）作为激活函数的前馈神经网络，在参数量充足的情况下具备模拟并逼近任意连续函数的能力，这一被称为“通用近似定理”的数学结论为现代深度学习的广泛应用奠定了坚实的基石。本实验旨在通过构建并训练基于 ReLU 激活函数的多层感知机（MLP）模型，对特定的非线性连续函数进行高度逼近拟合，并在实验范式上采取了双轨并行的严谨策略，即分别使用具有现代化自动求导机制的 TensorFlow 深度学习框架与纯粹依赖底层矩阵微积分运算的 NumPy 科学计算库来实现完全相同的网络拓扑结构。通过这种从高层调用到计算机底层微观链式法则推演的全链路对比，实验不仅直观验证了神经网络极其强大的非线性映射与特征拟合能力，更深刻揭示了深度学习模型在参数迭代优化与误差反向传播过程中的核心算理。

二、函数定义

在本次旨在验证神经网络泛化与逼近能力的深度学习拟合实验中，目标参考函数的合理构建是决定模型评估体系有效性的核心前提。为此放弃了单一的常规函数，精心构造了复合连续函数 $y = sin(x) + 0.5x$ 作为机器学习模型所要逼近的最终对象。

2.1 混合特征的深度融合

该复合函数巧妙地将代表局部非线性且具有高频周期震荡特征的三角函数分量 $sin(x)$，与代表全局稳定递增趋势的一次线性函数分量 $0.5x$ 进行了深度的数学叠加，从而在二维连续空间中构建出了一种既蕴含微观复杂波动又具备宏观确定性走向的混合数据分布形态。



图表 1 函数图像

2.2 非线性映射能力的严苛检验

这种刻意设计的非标准函数分布，使得网络模型不仅需要通过隐藏层的激活机制精准捕捉到数据在局部区间内的非线性曲折规律，还必须同时学习并拟合出贯穿全局的线性漂移趋势。这一设定跳出了单一规律的拟合舒适区，从而能够极其严苛且全面地检验深度神经网络模型面对复杂混合数据分布时的特征提取极限与高维映射能力。

三、数据采集与隔离规范

为确保模型训练过程的科学完备性以及最终模型评估指标的客观真实性，本实验在数据采集与预处理阶段严格贯彻了通用标准流程，特别是高度重视数据隔离与防泄露机制的系统性实施。

3.1 全量样本的无偏生成

首先，在既定的 $[-5,5]$ 这一涵盖完整具有正负对称性与周期性的定义域区间内，依托均匀随机采样机制，生成了包含 1250 个特征样本点的全量原始数据集，以此保障初始数据在特定分布空间内的无偏性与特征覆盖的充分性。

```python
# 数据采集
X_total = np.random.uniform(-5, 5, 1250).reshape(-1, 1)
y_total = target_function(X_total)
```

```
(.venv) PS C:\2351407-vscode\deep_learning\homework\chap_4\FunctionFitting> python generateDataset.py
数据集采集与切分完毕！
全量数据集: 1250 条
训练集 (80%): 1000 条 (已保存至 data/train.csv)
测试集 (20%): 250 条 (已保存至 data/test.csv)
```

图表 2 数据集生成

```csv
x,y
-4.542579661874871,-1.2856728220032885
-2.3193635917712374,-1.8923465211707016
-4.087938969513097,-1.2326846907816957
-3.2662640470524518,-1.5087833395062464
3.3493023679929905,1.4684317990377433
1.636711653626481,1.8161841981082771
-3.794331142227212,-1.289801370907487
-2.1276100834591825,-1.9127483376370193
```

图表 3 数据集示例

3.2 基于无放回抽样的绝对互斥切分

随后，为了模拟真实业务场景中的未知数据环境，借助无放回抽样技术对上述全量数据集进行了物理层面上的绝对互斥切分，将其严格划分为占比百分之八十的训练集与占比百分之二十的独立测试集。这种彻底阻断了训练信息流向测试环节的严密隔离机制，从根本上杜绝了因模型死记硬背训练样本而引发的数据泄露隐患，确立了测试集作为未见数据的绝对纯洁性，从而保证了最终测试阶段所产生的各项误差指标能够最真实、最客观地反映出模型的泛化预测水平。

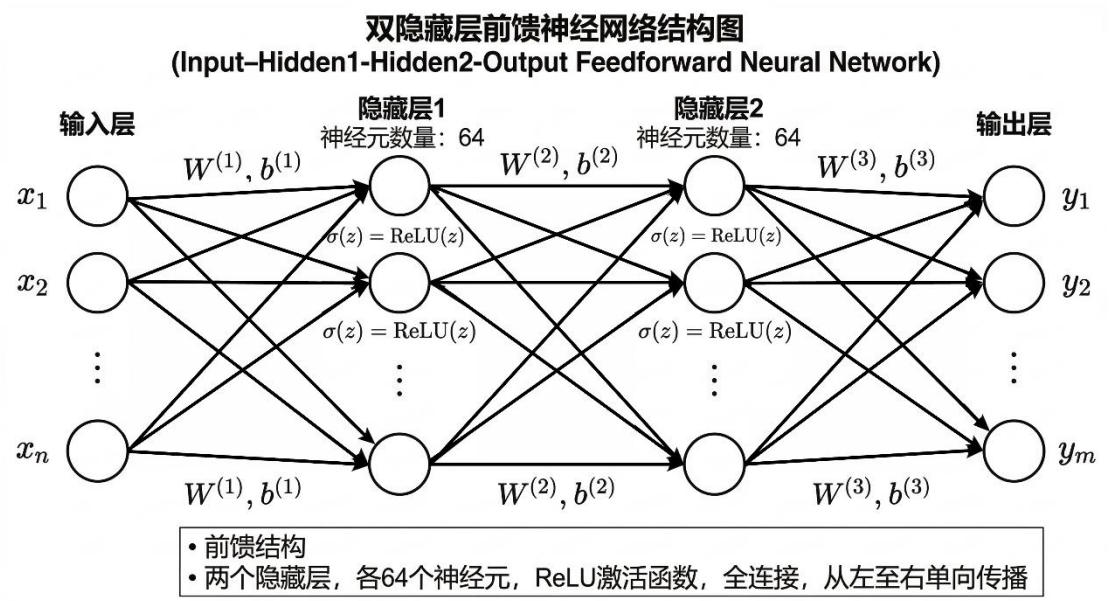
```
# 使用 train_test_split 进行严格的互斥切分
X_train, X_test, y_train, y_test = train_test_split(
    X_total, y_total, test_size=0.2, random_state=42
)
```

四、模型描述

针对上述复杂的混合非线性函数拟合需求，本实验在网络宏观拓扑结构的设计与微观代码逻辑的实现上采取了严谨的双轨并行策略，不仅搭建了具备高度非线性表达能力的多层感知机架构，还通过底层演算与高层框架进行了对比实现。

4.1 拓扑结构搭建与非线性映射机理

在网络拓扑骨架上，构建了一个标准的“输入层-双隐藏层-输出层”前馈神经网络模型，信息仅从输入层经过隐藏层单向传播至输出层，无反馈或循环结构，其中的两层隐藏层为了确保具备极其充裕的特征表达空间而高密度地配置了 64 个独立神经元，并统一采用了受生物学神经元真实放电机制启发且在现代深度学习中占据主导地位的 ReLU 作为激活函数；该函数凭借其特有的单侧抑制与无界限宽容特性——即当输入信号大于零时予以原样无损输出，而当输入信号小于等于零时则直接阻断输出为零——不仅在数学机制上彻底规避了传统 Sigmoid 或 Tanh 激活函数在深层网络反向传播中极易引发的梯度消失与计算饱和瓶颈，更为整个原本仅能执行空间缩放与平移的线性仿射变换网络引入了至关重要的非线性表达能力，使得原本僵化枯燥的矩阵乘加运算得以通过多层级的嵌套，灵活组合出能够无限逼近并拟合任意复杂多维曲面的分段线性折线。



图表 4 网络架构

4.2 目标驱动函数与回归输出配置

在决定最终预测结果形态的输出层架构设计上，鉴于本次实验任务属于典型的连续实数值回归预测范畴而非离散概率分类，因此在架构末端摒弃了任何形式的激活函数束缚以完美保留网络输出的原始无界域，并选用均方误差（MSE）作为衡量模型全局预测值矩阵与客观真实值标签之间欧氏距离差异的标量损失函数，将其作为在庞大且复杂的参数空间中，引导模型权重沿误差梯度下降方向不断收敛的唯一指标与最高优化准则。

4.3 基于纯 NumPy 的底层微积分手工推演

在将上述理论架构转化为具体代码时，首先采取了纯 NumPy 手工推演的实现方案，不仅要利用矩阵乘法法则极其精确地构建出前向传播过程中各个神经元节点错综复杂的仿射变换逻辑，更要严格遵循微积分中的复合函数链式求导法则，自代表最终误差的损失函数输出层起，逐层向后以纯手工数学推演的方式精准计算出局部误差关于每一个权重矩阵与偏置向量的偏导数公式，最终依托标准的批量梯度下降（BGD）算法并在事先设定的静态全局学习率驱动下完成网络中每一个神经元连接权重的逆向微调与世代迭代更新。

```
class NumPyMLP:
    def __init__(self, input_dim=1, hidden_dim=64, output_dim=1,
lr=0.01):
        # 权重初始化
        self.W1 = np.random.randn(input_dim, hidden_dim) * 0.1
        self.b1 = np.zeros((1, hidden_dim))
```

```
        #W2、W3、b2、b3 初始类似
        self.lr = lr

    def relu(self, x):
        return np.maximum(0, x)

    def relu_deriv(self, x):
        return (x > 0).astype(float)

    def forward(self, X):
        self.X = X
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.relu(self.z2)
        self.z3 = np.dot(self.a2, self.W3) + self.b3
        return self.z3

    def backward(self, y_true, y_pred):
        m = y_true.shape[0]
        # 链式法则求导
        dz3 = 2 * (y_pred - y_true) / m
        dW3 = np.dot(self.a2.T, dz3)
        db3 = np.sum(dz3, axis=0, keepdims=True)

        da2 = np.dot(dz3, self.W3.T)
        dz2 = da2 * self.relu_deriv(self.z2)
        dW2 = np.dot(self.a1.T, dz2)
        db2 = np.sum(dz2, axis=0, keepdims=True)
        # da1、dz1、dW1、db1 处理与上类似

        # 梯度下降更新
        self.W3 -= self.lr * dW3  # 其他参数处理类似

    def train(self, X, y, epochs=3000):
        loss_history = []
        for epoch in range(epochs):
            # 前向传播、后向传播、计算 loss
        return loss_history
```

4.4 基于 TensorFlow 的计算图抽象与自动求导

与纯手工底层演算的形成鲜明对比的，是基于 TensorFlow 框架的实现路线，它充分展

现了现代深度学习底层架构的高度抽象与宏观算力优化的效率；该框架通过其内部深度封装的动态计算图机制与极其强大的自动求导引擎，能够在模型执行前向推理计算的过程中，于底层静默、连续且精确地追踪并记录下所有的操作算子节点、多维张量流向与各层激活状态，进而通过编译器在瞬间自动生成与之严密对应的复杂反向传播求导通路。

```
    # 构建与训练模型
    model = models.Sequential([
        Input(shape=(1,)),
        layers.Dense(64, activation='relu'),
        layers.Dense(64, activation='relu'),
        layers.Dense(1)
    ])
    model.compile(optimizer='adam', loss='mse', metrics=['mae'])
    history = model.fit(X_train, y_train, epochs=200, batch_size=32,
verbose=1)
```

同时，该路径配合了收敛速度更为迅捷、能够巧妙利用梯度的一阶动量与二阶动量来为网络中数以万计的参数自适应动态调节专属学习率的 Adam 优化器，使得开发者得以从繁杂、枯燥且极易产生矩阵维度失配错误的底层微积分演算中解放，从而将宝贵的核心系统算力与资源统筹聚焦于网络宏观架构设计、超参数调优等方面之上。

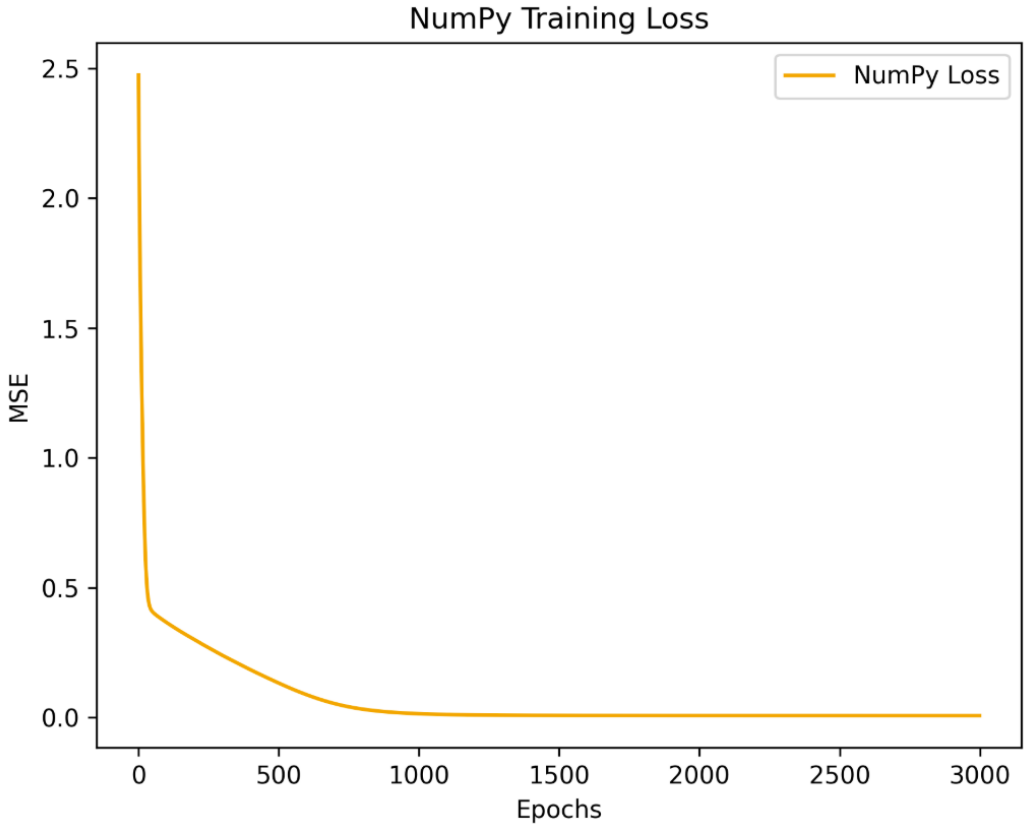
五、拟合效果对比与实验结论

结合实验最终导出的可视化结果图表（numpy_results.png 与 tf_results.png）以及终端所输出的独立测试集评估指标进行横向与纵向的深度比对分析，能够从训练动态、视觉拟合、量化指标以及底层哲理四个维度，得出以下结论。

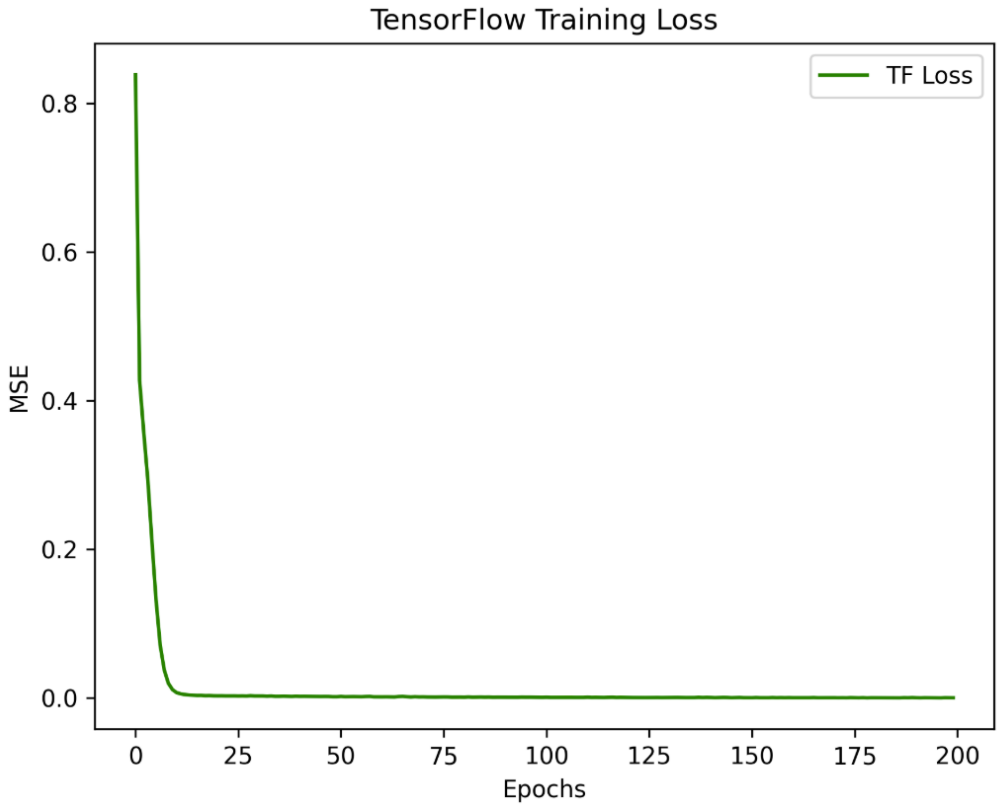
5.1 高度一致的最优化收敛轨迹

在反映模型内部参数寻优进程的损失函数（Loss）层面可以极为清晰地观察到，无论是依托纯 NumPy 从零构建的手工模型，还是由 TensorFlow 驱动的网络，两者在历经各自所需的成百上千次充分迭代后，其训练误差曲线均呈现出符合最优化动力学理论预期的演化轨迹：即在训练初期伴随巨大梯度势能呈现出指数级的陡峭下降，并在训练中后期随着参数向量逐渐逼近多维误差曲面的谷底而稳健地收敛，最终平滑地维持在一个极低误差水平的健康

动态平稳态势之中。



图表 5 numpy 训练损失函数



图表 6 tf 训练损失曲线

但由于底层优化算法与算力调度机制的显著差异，二者各自收敛所需的迭代次数相距甚大。TensorFlow 框架得益于带有动量加速与自适应学习率调节功能的 Adam 优化器，能够在仅仅迭代数十次左右的周期内迅速完成收敛，而仅依赖基础批量梯度下降（BGD）且采用静态学习率的纯 NumPy 手工模型，则需历经数百次的迭代才能达到稳态。

可以观察到 numpy 的训练损失函数曲线存在一个较为明显的拐点，在经过拐点前损失下降十分剧烈，但在拐点之后损失下降趋于平缓，而 tensorflow 框架的损失函数图中的拐点并不明显。该现象的出现主要有以下几个原因：

①多层神经网络的参数空间极其庞大且充满着非凸特性，其误差曲面并非一个完美的平滑漏斗，而是布满了大量的平坦区域或鞍点（在某些维度上梯度为零，但在其他维度上仍有下降空间的点）；当模型参数在固定学习率的驱动下漫游至这些平坦区域时，计算出的梯度向量几乎趋近于零，导致权重更新陷入停滞，Loss 曲线在视觉上便呈现出一段漫长的横向平缓期；而随着成百上千次的微小累积更新，参数一旦极为偶然地跨越了该平坦区域的边缘并寻找到了一条陡峭的下降峡谷，梯度便会瞬间激增，使得 Loss 曲线以断崖式的拐点迅速向下坠落。

②深度学习的频谱偏差理论指出，神经网络在拟合复杂函数时总是倾向于优先学习低频的、全局的简单主导模式，随后再去死磕高频的、局部的复杂震荡细节；对应到设定的目标函数中，NumPy 模型在训练初期会以极快的速度拟合出占据主导地位的低频线性趋势（$0.5x$），这对应着 Loss 曲线最开始的快速下降；但当线性部分拟合完毕后，模型遭遇了高频非线性项（$sin(x)$）的拟合瓶颈，导致 Loss 曲线进入停滞期，直到权重矩阵经过长时间的内部重组，终于在某个节点顿悟并成功锁定了正弦波的周期震荡特征时，便会引发 Loss 曲线的第二次破局与大幅度拐点下降。

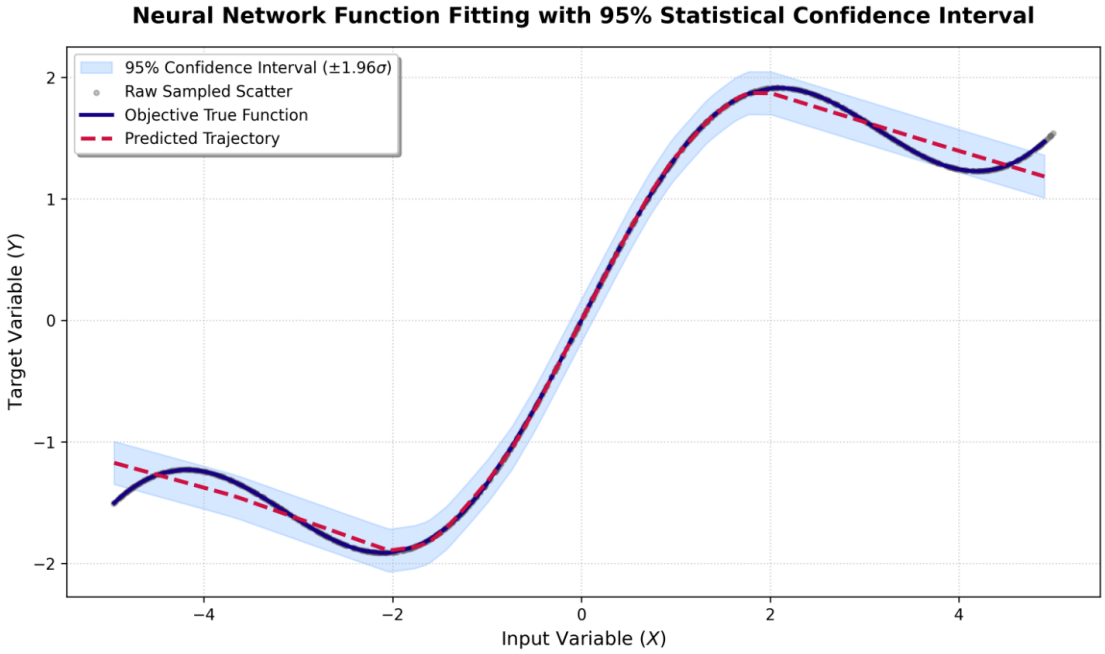
③ReLU 激活函数的本质是通过输入信号的正负来决定神经元的开启或关闭，在漫长的平缓期内，网络中可能存在大量处于休眠状态（输出为零，梯度阻断）的死神经元；随着前置权重的不断微调，一旦某些关键输入特征的仿射变换结果跨越了零点阈值，这些原本沉睡的神经元便会被成批地群体性唤醒，使得网络整体的非线性表达容量与拟合维度在瞬间得到爆炸性跃升，从而直接导致预测误差在几个迭代内出现拐点式的骤降。

④这一现象之所以在纯 NumPy 的手工模型中表现得如此明显，主要根源在于使用的批量梯度下降法（BGD）且辅以绝对静态的全局学习率；与之形成鲜明对照的是，TensorFlow 框架内部所默认使用的 Adam 优化器，能够凭借一阶动量（惯性保持下降方向）与二阶动量

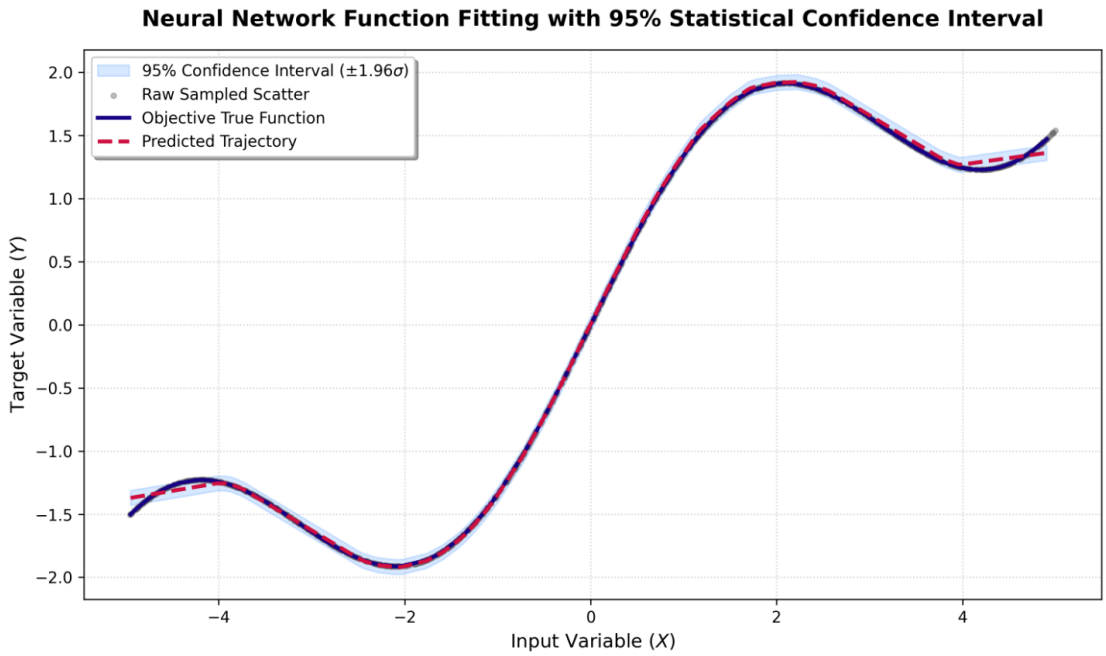
（自适应缩放平缓维度的步长）平滑跨越引发停滞的鞍点与平原，因此在 TF 的 Loss 曲线上，这种由于地形崎岖而导致的突兀拐点会被极其平滑且高速的收敛轨迹所掩盖。

5.2 解构复杂规律的视觉映射

聚焦于直接验证模型非线性泛化能力的函数拟合效果可视化对比区域，由双隐藏层多层感知机通过前向推理所生成的预测虚线，几乎覆盖并紧密贴合了代表客观真实物理规律的连续蓝色实线。同时原始采样分布特征的离散散点被囊括在预测轨迹的合理统计置信区间内部。这直观有力地印证了，经过充分训练的网络拓扑结构已经凭借其内部神经元激活状态的组合，解构并精准捕捉到了混合目标函数背后隐藏的“局部高频非线性震荡叠加全局稳定线性递增”数理逻辑。



图表 7 numpy 拟合图



图表 8 tf 拟合图

进一步对 TensorFlow 与 NumPy 两者的拟合图像展开微观层面的细致比对，则能够清晰地察觉到底层优化算法差异在预测轨迹上的体现。得益于 Adam 优化器在动量加速与自适应多维步长调节上的技术优势，TensorFlow 框架所生成的预测虚线呈现出较高的视觉平滑度，其对正弦波峰谷等高曲率极值区域的动态追踪更为接近；相较之下，纯 NumPy 手工模型由于仅依赖静态学习率的朴素批量梯度下降（BGD），其预测轨迹在局部拐点处更为明显地暴露出了 ReLU 激活函数本质上作为“分段线性逼近”的物理拼接痕迹，在个别高频震荡极值点的振幅包裹上也显露出微小的迟滞与边际误差。然而，尽管在微观特征的极致平滑度上存在着这种由算力引擎与优化器代差所带来的细微分野，但两者在宏观上均拟合了复杂的非线性函数，殊途同归地彰显了多层神经网络强大的逼近定律。

5.3 卓越泛化性能的证实

进一步比较终端最终输出的客观定量评估结果可知，在完全剥离了训练环境的独立验证测试集上，两种截然不同的代码实现路径均得到了较低数量级且稳定的测试集均方误差（MSE）与平均绝对误差（MAE）；这一测算不仅从数据层面再次证实了，具备充足宽度参数的双层 ReLU 神经网络在处理连续实变函数逼近任务时所展现出的卓越泛化推理性能，更从根本上排除了模型发生“过拟合”退化现象的理论可能。

```
Epoch 200/3000 - loss: 0.3005 - mae: 0.4893
Epoch 300/3000 - loss: 0.2397 - mae: 0.4382
Epoch 400/3000 - loss: 0.1843 - mae: 0.3850
Epoch 500/3000 - loss: 0.1334 - mae: 0.3268
Epoch 600/3000 - loss: 0.0882 - mae: 0.2627
Epoch 700/3000 - loss: 0.0539 - mae: 0.1989
Epoch 800/3000 - loss: 0.0325 - mae: 0.1466
Epoch 900/3000 - loss: 0.0209 - mae: 0.1130
Epoch 1000/3000 - loss: 0.0149 - mae: 0.0953
Epoch 1100/3000 - loss: 0.0117 - mae: 0.0848
Epoch 1200/3000 - loss: 0.0100 - mae: 0.0779
Epoch 1300/3000 - loss: 0.0091 - mae: 0.0733
Epoch 1400/3000 - loss: 0.0085 - mae: 0.0700
Epoch 1500/3000 - loss: 0.0082 - mae: 0.0677
Epoch 1600/3000 - loss: 0.0080 - mae: 0.0660
Epoch 1700/3000 - loss: 0.0079 - mae: 0.0647
Epoch 1800/3000 - loss: 0.0078 - mae: 0.0637
Epoch 1900/3000 - loss: 0.0078 - mae: 0.0629
Epoch 2000/3000 - loss: 0.0077 - mae: 0.0623
Epoch 2100/3000 - loss: 0.0077 - mae: 0.0618
Epoch 2200/3000 - loss: 0.0077 - mae: 0.0614
Epoch 2300/3000 - loss: 0.0077 - mae: 0.0611
Epoch 2400/3000 - loss: 0.0077 - mae: 0.0608
Epoch 2500/3000 - loss: 0.0076 - mae: 0.0606
Epoch 2600/3000 - loss: 0.0076 - mae: 0.0604
Epoch 2700/3000 - loss: 0.0076 - mae: 0.0602
Epoch 2800/3000 - loss: 0.0076 - mae: 0.0600
Epoch 2900/3000 - loss: 0.0076 - mae: 0.0598
Epoch 3000/3000 - loss: 0.0076 - mae: 0.0597
Numpy 训练结束
------------------------------
NumPy 测试集最终评估:
Loss (MSE): 0.0073
MAE (平均绝对误差): 0.0597
------------------------------
NumPy 训练完成! 结果和图表已保存至 results/numpy_results.png
```

图表 9 numpy 输出

```
Epoch 89/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0011 - mae: 0.0185
Epoch 90/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0013 - mae: 0.0243
Epoch 91/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0011 - mae: 0.0210
Epoch 92/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 9.7562e-04 - mae: 0.0190
Epoch 93/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0012 - mae: 0.0235
Epoch 94/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0011 - mae: 0.0204
Epoch 95/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0012 - mae: 0.0226
Epoch 96/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0011 - mae: 0.0203
Epoch 97/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.0010 - mae: 0.0194
Epoch 98/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 8.8725e-04 - mae: 0.0175
Epoch 99/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 9.5764e-04 - mae: 0.0177
Epoch 100/100
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 9.4902e-04 - mae: 0.0184
TensorFlow 训练结束
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
------------------------------
TensorFlow 测试集最终评估:
Loss (MSE): 0.0008
MAE (平均绝对误差): 0.0197
------------------------------
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
结果和图表已保存至 results/tf_results.png
```

图表 10 tf 输出

然而，在共性的卓越表现之下，底层优化算法的技术差异依然在数据精度层面所显现。纯 NumPy 手工模型最终收敛于 0.0597 的均方误差与 0.0073 的平均绝对误差，而由 TensorFlow 框架驱动的模型则凭借 Adam 优化器在动量加速与自适应多维步长调节上的优势，将这两项核心误差指标进一步压缩到了 0.0197 与 0.0008 的更微小量级。这种极其显著的数值差异再次揭示出，尽管基础朴素的 BGD 算法能够在宏观方向上引导模型参数走向收敛，但现代深度学习框架内部高度封装的动态计算图与高级自适应优化引擎，能够帮助模型在错综复杂的高维非凸误差曲面中更为敏锐且精准地规避局部极小值与平坦鞍点，最终稳健地收敛于更接近全局最优解的区域内。

5.4 数理同归

纵观本次实验，这一最终的对比结果深刻地揭示了：尽管手写逻辑的代码实现路径与现代深度学习计算框架之间，存在着宏观调用与微观多维矩阵偏导数演算在抽象层级上的巨大鸿沟，但在正确误差反向传播的数学推导理论支撑与约束之下，无论是依赖于系统底层的自动求导封装还是依靠于人类脑力的手工验算，模型最终都必定能抵达系统误差收敛的最优稳态。

九、参考文献

[1] G. Cybenko. 1989. Approximation by superpositions of a sigmoidal function.

[2] K. Hornik, M. Stinchcombe, and H. White. 1989. Multilayer feedforward networks are universal approximators.

[3] Moshe Leshno, et al. 1993. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function

[4] Vinod Nair and Geoffrey E. Hinton. 2010. Rectified linear units improve restricted boltzmann machines.

[5] Xavier Glorot, Antoine Bordes, Yoshua Bengio. 2011. Deep Sparse Rectifier Neural Networks. PMLR 15:315-323.”